

```
In [1]: import pandas as pd
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt
from tensorflow import keras
from tensorflow.keras import layers

In [2]: dataset = pd.read_csv('mpg.csv')

In [3]: dataset

Out[3]:
```

	MPG	Cylinders	Displacement	Horsepower	Weight	Acceleration	Model Year	Origin
0	18.0	8	307.0	130.0	3504.0	12.0	70	1
1	15.0	8	350.0	165.0	3693.0	11.5	70	1
2	18.0	8	318.0	150.0	3436.0	11.0	70	1
3	16.0	8	304.0	150.0	3433.0	12.0	70	1
4	17.0	8	302.0	140.0	3449.0	10.5	70	1
...	...	...	...	...	...	...	...	...
393	27.0	4	140.0	86.0	2790.0	15.6	82	1
394	44.0	4	97.0	52.0	2130.0	24.6	82	2
395	32.0	4	135.0	84.0	2295.0	11.6	82	1
396	28.0	4	120.0	79.0	2625.0	18.6	82	1
397	31.0	4	119.0	82.0	2720.0	19.4	82	1

398 rows × 8 columns

```
In [4]: dataset.isna().sum()

Out[4]: MPG          0
Cylinders        0
Displacement     0
Horsepower       6
Weight           0
Acceleration     0
Model Year       0
Origin           0
dtype: int64

In [5]: dataset.dropna(inplace=True)

In [6]: dataset.isna().sum()

Out[6]: MPG          0
Cylinders        0
Displacement     0
Horsepower       0
Weight           0
Acceleration     0
Model Year       0
Origin           0
dtype: int64

In [7]: dataset

Out[7]:
```

	MPG	Cylinders	Displacement	Horsepower	Weight	Acceleration	Model Year	Origin
0	18.0	8	307.0	130.0	3504.0	12.0	70	1
1	15.0	8	350.0	165.0	3693.0	11.5	70	1
2	18.0	8	318.0	150.0	3436.0	11.0	70	1
3	16.0	8	304.0	150.0	3433.0	12.0	70	1
4	17.0	8	302.0	140.0	3449.0	10.5	70	1
...	...	...	...	...	...	...	...	...
393	27.0	4	140.0	86.0	2790.0	15.6	82	1
394	44.0	4	97.0	52.0	2130.0	24.6	82	2
395	32.0	4	135.0	84.0	2295.0	11.6	82	1
396	28.0	4	120.0	79.0	2625.0	18.6	82	1
397	31.0	4	119.0	82.0	2720.0	19.4	82	1

392 rows × 8 columns

```
In [8]: dataset.Origin.unique()

Out[8]: array([1, 3, 2])

In [9]: dataset['Origin']=dataset['Origin'].map({1:'USA',2:'Europe',3:'Japan'})

In [10]: dataset.head(5)

Out[10]:
```

	MPG	Cylinders	Displacement	Horsepower	Weight	Acceleration	Model Year	Origin
0	18.0	8	307.0	130.0	3504.0	12.0	70	USA
1	15.0	8	350.0	165.0	3693.0	11.5	70	USA
2	18.0	8	318.0	150.0	3436.0	11.0	70	USA
3	16.0	8	304.0	150.0	3433.0	12.0	70	USA
4	17.0	8	302.0	140.0	3449.0	10.5	70	USA

```
In [11]: dataset = pd.get_dummies(dataset, columns=['Origin'])

In [12]: dataset.head(5)
```

Out[12]:

	MPG	Cylinders	Displacement	Horsepower	Weight	Acceleration	Model Year	Origin_Europe	Origin_Japan	Origin_USA
0	18.0	8	307.0	130.0	3504.0	12.0	70	False	False	True
1	15.0	8	350.0	165.0	3693.0	11.5	70	False	False	True
2	18.0	8	318.0	150.0	3436.0	11.0	70	False	False	True
3	16.0	8	304.0	150.0	3433.0	12.0	70	False	False	True
4	17.0	8	302.0	140.0	3449.0	10.5	70	False	False	True

In [13]:

dataset.describe()

Out[13]:

	MPG	Cylinders	Displacement	Horsepower	Weight	Acceleration	Model Year
count	392.000000	392.000000	392.000000	392.000000	392.000000	392.000000	392.000000
mean	23.445918	5.471939	194.411990	104.469388	2977.584184	15.541327	75.979592
std	7.805007	1.705783	104.644004	38.491160	849.402560	2.758864	3.683737
min	9.000000	3.000000	68.000000	46.000000	1613.000000	8.000000	70.000000
25%	17.000000	4.000000	105.000000	75.000000	2225.250000	13.775000	73.000000
50%	22.750000	4.000000	151.000000	93.500000	2803.500000	15.500000	76.000000
75%	29.000000	8.000000	275.750000	126.000000	3614.750000	17.025000	79.000000
max	46.600000	8.000000	455.000000	230.000000	5140.000000	24.800000	82.000000

In [14]:

train_dataset = dataset.sample(frac=0.8, random_state=2)
test_dataset = dataset.drop(train_dataset.index)

In [15]:

train_features = train_dataset.copy()
test_features = test_dataset.copy()

train_labels = train_features.pop('MPG')
test_labels=test_features.pop('MPG')

In [16]:

normalizer = tf.keras.layers.Normalization(axis=-1)
normalizer.adapt(np.array(train_features))

In [17]:

def build_and_compile(norm):

 model = keras.Sequential([
 norm,
 layers.Dense(128, activation='relu'),
 layers.Dense(128, activation='relu'),
 layers.Dense(1)

])
 model.compile(loss='mean_absolute_error',optimizer=tf.keras.optimizers.Adam(0.01))

 return model

In [18]:

dnn_model = build_and_compile(normalizer)

In [19]:

def plot_loss(history):
 plt.plot(history.history['loss'], label='loss')
 plt.plot(history.history['val_loss'], label='val_loss')
 plt.ylim([0, 10])
 plt.xlabel('Epoch')
 plt.ylabel('Error [MPG]')
 plt.legend()
 plt.grid(True)

In []:

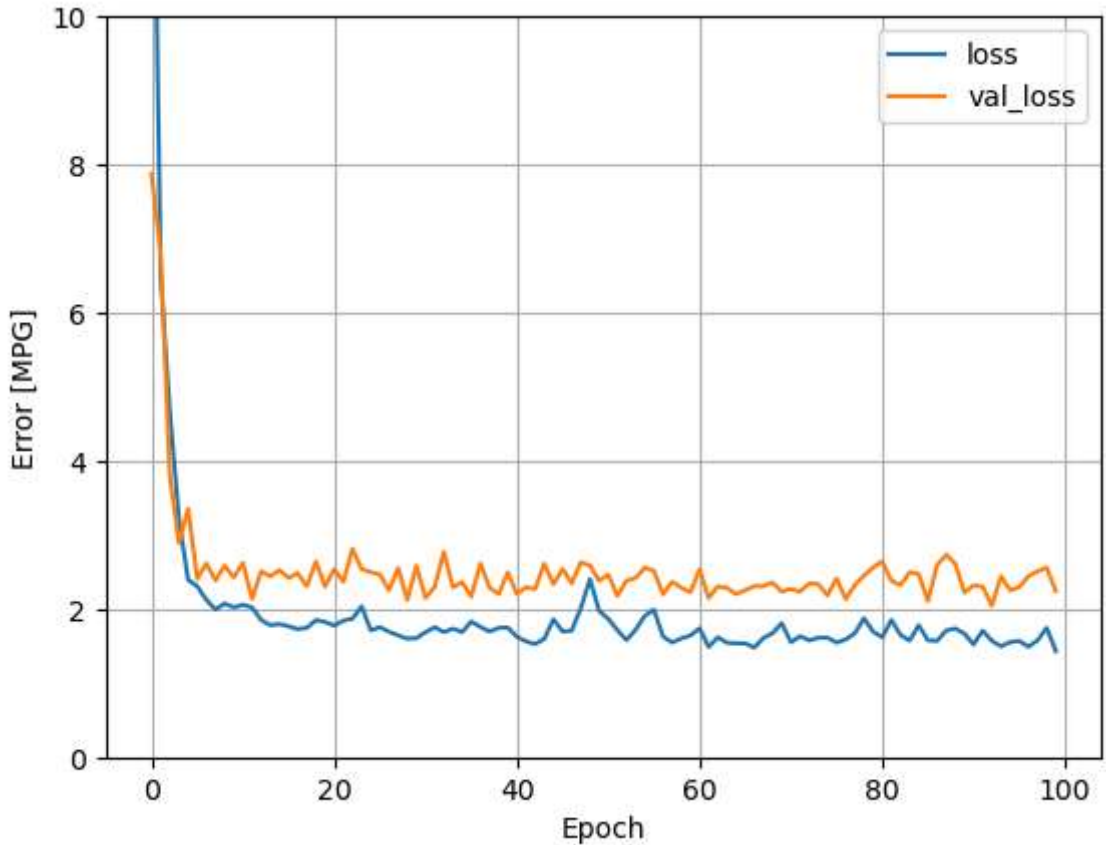
In []:

In [20]:

history = dnn_model.fit(
 train_features,
 train_labels,
 validation_split=0.2,
 verbose=0,
 epochs=100,
)

In [21]:

plot_loss(history)



In [22]:

dnn_model.summary()

Model: "sequential"

Layer (type)	Output Shape	Param #
normalization (Normalization)	(None, 9)	19
dense (Dense)	(None, 128)	1,280
dense_1 (Dense)	(None, 128)	16,512
dense_2 (Dense)	(None, 1)	129

Total params: 53,784 (210.10 KB)
Trainable params: 17,921 (70.00 KB)
Non-trainable params: 19 (80.00 B)
Optimizer params: 35,844 (140.02 KB)

```
In [23]: mae = dnn_model.evaluate(test_features,test_labels)

3/3 ----- 0s 10ms/step - loss: 1.7693

In [24]: print (f'The MAE for your model is {mae} MPG')

The MAE for your model is 1.7692633867263794 MPG

In [ ]:
```